



Design and Analysis of Algorithms

Chapter 5 – Divide and Conquer

Divide and Conquer

- ❖ Divide-and-conquer, breaks a problem into subproblems that are similar to the original problem, recursively solves the subproblems, and finally combines the solutions to the subproblems to solve the original problem.
- ❖ Divide-and-conquer solves subproblems recursively, each subproblem must be smaller than the original problem, and there must be a base case for subproblems. You should think of a divide-and-conquer strategy as having three parts:

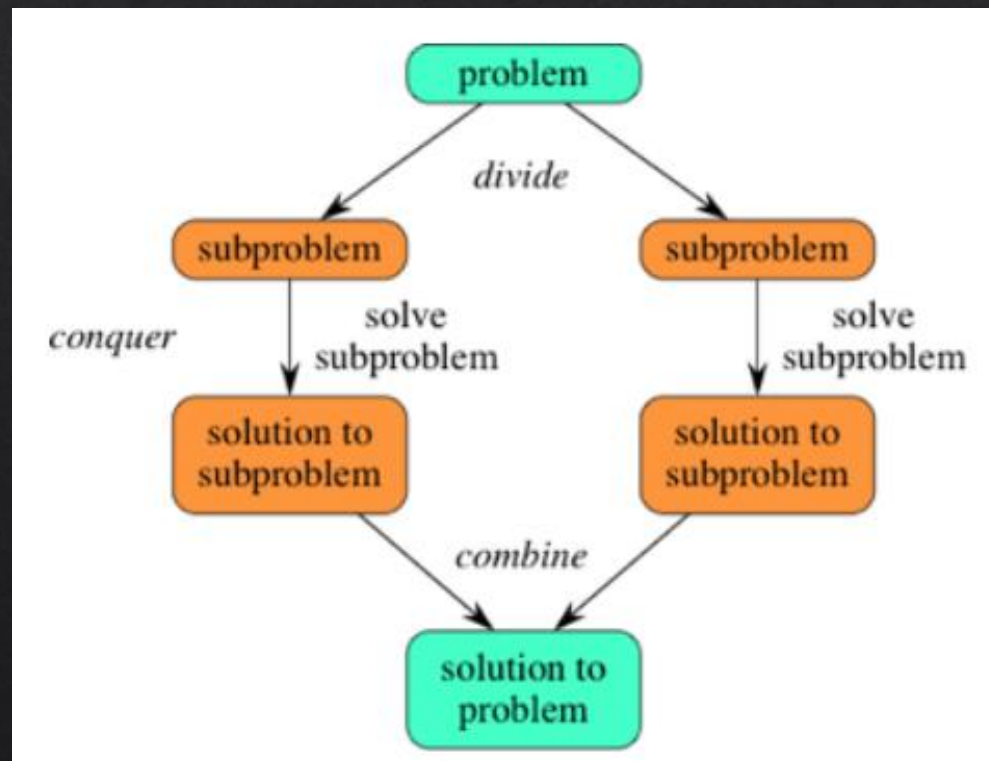
Cont..

- ◆ **Divide** the problem into a number of subproblems that are smaller instances of the same problem.
- ◆ **Conquer** the subproblems by solving them recursively. If they are small enough, solve the subproblems as base cases.
- ◆ **Combine** the solutions to the subproblems into the solution for the original problem.

You can easily remember the steps of a divide-and-conquer algorithm as divide, conquer, combine.

Cont..

- ◆ Here's how to view one step, assuming that each divide step creates two subproblems (though some divide-and-conquer algorithms create more than two):



Cont..

The following are some standard algorithms that follow Divide and Conquer algorithm:

1. **Quick Sort**: is a sorting algorithm. The algorithm picks a pivot element and rearranges the array elements so that all elements smaller than the picked pivot element move to the left side of the pivot, and all greater elements move to the right side. Finally, the algorithm recursively sorts the subarrays on the left and right of the pivot element.
2. **Merge Sort**: is also a sorting algorithm. The algorithm divides the array into two halves, recursively sorts them, and finally merges the two sorted halves.

Cont..

The following are some standard algorithms that follow Divide and Conquer algorithm:

1. **Closest Pair of Points**: The problem is to find the closest pair of points in a set of points in the x-y plane. The problem can be solved in $O(n^2)$ time by calculating the distances of every pair of points and comparing the distances to find the minimum. The Divide and Conquer algorithm solves the problem in $O(N \log N)$ time.
2. **Strassen's Algorithm**: is an efficient algorithm to multiply two matrices. A simple method to multiply two matrices needs 3 nested loops and is $O(n^3)$. Strassen's algorithm multiplies two matrices in $O(n^{2.81})$ time.

Strassen's Algorithm

- ◆ **The Strassen Algorithm**, named after **Volker Strassen**, is an algorithm for **matrix multiplication**.
- ◆ **Volker Strassen** first published this algorithm in 1969 and thereby proved that the n^3 general matrix multiplication algorithm was not optimal
- ◆ It is faster than the standard matrix multiplication algorithm for large matrices, with a better asymptotic complexity.
- ◆ Useful in practice for large matrices, but would be slower than the fastest known algorithms for extremely large matrices.

Cont..

- ◇ The goal of matrix multiplication is to calculate the matrix product **C** = **AB**.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix},$$

- ◇ **8** multiplications of matrix blocks are needed to calculate the **C** matrices, when using standard matrix multiplication.

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}$$

- ◇ We have 8 products and 4 sums!

Cont..

Here is the Standard Matrix Multiplication Algorithms:

```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j < n; j++)  
    {  
        C[i][j] = 0;  
        for (k = 0; k < n; k++)  
            C[i][j] += A[i][k] * B[k][j];  
    }  
}
```

- ◆ It's complexity is $O(n^3)$, since it contains three nested loops.

Cont..

- ◆ The **Strassen algorithm** defines instead new matrices:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22});$$

$$M_2 = (A_{21} + A_{22})B_{11};$$

$$M_3 = A_{11}(B_{12} - B_{22});$$

$$M_4 = A_{22}(B_{21} - B_{11});$$

$$M_5 = (A_{11} + A_{12})B_{22};$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12});$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22}),$$

- ◆ Using only **7** multiplications instead of **8**

Cont..

◊ We may now express the C in terms of M_k :

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{bmatrix}$$

◊ At this time, the complexity is $\mathcal{O}(n^{2.81})$.

Understanding Strassen's Approach

Divide-and-Conquer

- The Strassen Algorithm employs a divide-and-conquer strategy. It breaks down the original matrices into smaller sub-matrices, recursively applies multiplication to these sub-matrices, and finally combines the results to obtain the final product.
- This process efficiently reduces the overall number of multiplications.

Seven Intermediate Products

- Instead of performing eight multiplications as in standard matrix multiplication, Strassen's algorithm cleverly defines seven intermediate products (M1 to M7).
- These products are calculated using specific combinations of sub-matrix elements from the original matrices. By strategically combining these products, the final matrix product is derived.

Example: Multiplication of 2x2 Matrices

Matrix A

$$\begin{pmatrix} \mathbf{2}_{(A11)} & \mathbf{3}_{(A12)} \\ \mathbf{1}_{(A21)} & \mathbf{4}_{(A22)} \end{pmatrix}$$



Matrix B

$$\begin{pmatrix} \mathbf{1}_{(B11)} & \mathbf{6}_{(B12)} \\ \mathbf{4}_{(B21)} & \mathbf{2}_{(B22)} \end{pmatrix}$$

Standard Matrix Multiplication (8 Multiplications). Strassen?

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}$$

Computing Strassen's Products: Step 1

M1

$$M1 = (A11 + A22) * (B11 + B22)$$

M2

$$M2 = (A21 + A22) * B11$$

M3

$$M3 = A11 * (B12 - B22)$$

M4

$$M4 = A22 * (B21 - B11)$$

M5

$$M5 = (A11 + A12) * B22$$

M6

$$M6 = (A21 - A11) * (B11 + B12)$$

M7

$$M7 = (A12 - A22) * (B21 + B22)$$

Calculating the Intermediate Products: -

18

M1

$$\mathbf{M1} = (2 + 4) * (1 + 2) = 6 * 3 = 18$$

5

M2

$$\mathbf{M2} = (1 + 4) * 1 = 5 * 1 = 5$$

8

M3

$$\mathbf{M3} = 2 * (6 - 2) = 2 * 4 = 8$$

12

M4

$$\mathbf{M4} = 4 * (4 - 1) = 4 * 3 = 12$$

10

M5

$$\mathbf{M5} = (2 + 3) * 2 = 5 * 2 = 10$$

-7

M6

$$\mathbf{M6} = (1 - 2) * (1 + 6) = (-1) * 7 = -7$$

-6

M7

$$\mathbf{M7} = (3 - 4) * (4 + 2) = (-1) * 6 = -6$$

Step 2: Combining the Products



C11

$$C11 = M1 + M4 - M5 + M7$$

$$C11 = 18 + 12 - 10 + (-6) = \mathbf{14}$$



C12

$$C12 = M3 + M5$$

$$C12 = 8 + 10 = \mathbf{18}$$



C21

$$C21 = M2 + M4$$

$$C21 = 5 + 12 = \mathbf{17}$$



C22

$$C22 = M1 - M2 + M3 + M6$$

$$C22 = 18 - 5 + 8 + (-7) = \mathbf{14}$$

The Resulting Product Matrix

The product of A x B, computed using the Strassen Algorithm, is:

$$\begin{pmatrix} 14_{(C11)} & 18_{(C12)} \\ 17_{(C21)} & 14_{(C22)} \end{pmatrix}$$

Efficiency!

- By effectively reducing the number of multiplications from eight (in standard matrix multiplication) to seven, the Strassen Algorithm demonstrates significant efficiency, particularly for larger matrices.
- This efficiency translates to reduced computational time and resources, making it valuable in diverse applications like computer graphics, scientific simulations, and machine learning algorithms.

THE END

THANK YOU!